Software Requirements Specification: The Listener Module – Project L.O.V.E.

1. Executive Summary and Strategic Architecture

1.1 The Theoretical Imperative: Beyond Cartesian Affect

The domain of Affective Computing has historically been constrained by a reliance on two-dimensional models of human emotion, primarily the Circumplex Model of Affect established by James Russell in 1980. This traditional framework maps emotional experience onto a Cartesian grid defined by Valence (the hedonic tone of pleasure versus displeasure) and Arousal (the physiological state of activation versus deactivation). While this model has proven sufficient for basic sentiment analysis and binary mood tracking, it suffers from a critical semantic collapse when applied to the complex, relational nature of the human experience as described in contemporary psychological literature.

Project L.O.V.E. (Listener - Observer - Versor - Experience) represents a fundamental paradigm shift in this field. It rejects the static, 2D grid in favor of a dynamic, three-dimensional conceptual space that integrates the rigorous qualitative research of Dr. Brené Brown's *Atlas of the Heart*. The central thesis of this project is that emotions are not fixed points to be inhabited, but dynamic vectors of orientation. To map the "trajectory of the human soul," the system introduces a third axis: **Connection**.

This new **VAC Model (Valence, Arousal, Connection)** allows for the mathematical differentiation of emotional states that appear identical in traditional models. For instance, "Pity" and "Compassion" may share similar coordinates in Valence (negative/sorrowful) and Arousal (low/moderate), but they are orthogonal in their relational orientation. Pity is defined by separation and a sense of "feeling for" rather than "feeling with," resulting in a negative Connection vector. Compassion is defined by shared humanity and relational alignment, resulting in a positive Connection vector.[1]

1.2 The Listener: The Sensory Cortex of the System

This Software Requirements Specification (SRS) is dedicated to the **Listener** module, the first and most critical component of the L.O.V.E. Stack. If the Versor is the mathematical engine that calculates the "work" of emotional rotation using quaternions, and the Observer is the memory that persists the trajectory, the Listener is the sensory cortex that perceives the world.

The Listener's mandate is to ingest unstructured, multi-modal human signals—specifically voice and text—and transmute them into precise, normalized mathematical vectors (v_x, v_y, v_z) and semantic classifications. This is not a trivial transcription task; it is a complex extraction process that must discern the subtle "Connection" (z-axis) signals often hidden between the lines of spoken language.

The Listener must operate within a "Split-Brain" architecture:

1. **The Reflex Loop (Edge):** A hyper-fast, on-device transcription layer using quantized models (Whisper Tiny/Base) to provide immediate visual feedback to the user, ensuring the application feels responsive and "alive" (latency < 200ms).
2. **The Reflective Loop (Cloud):** A deep-processing layer using server-grade models (Whisper Large-v3) and Large Language Models (LLMs) to perform the heavy lifting of semantic vectorization, "Atlas" mapping, and PII (Personally Identifiable Information) sanitization (latency < 3000ms).

This document serves as the exhaustive technical blueprint for the Listener module. It is designed to be ingested by an AI coding assistant (Claude) to generate the production-ready code for the ingestion pipeline, the semantic processing engine, and the API interfaces that bind the L.O.V.E. system together.

---

2. Theoretical Framework: The Digital Atlas of the Heart

To build the Listener, we must first define the ontology it is listening for. The system relies on the taxonomy of 87 emotions identified in *Atlas of the Heart*.[2] The Listener's primary semantic task is to classify user input into one of these 87 states and derive the corresponding VAC vector.

2.1 The Dimensions of the Soul (VAC Model)

The Listener must extract three scalar values from every input, normalized to a range of $[-1.0, +1.0]$.

- **Valence (x-axis):** The intrinsic attractiveness (positive) or aversiveness (negative) of the state.
    - *Range:* -1.0 (Despair, Anguish) to $+1.0$ (Joy, Ecstasy).
    - *Extraction Cue:* Sentiment magnitude, adjective polarity (e.g., "wonderful" vs. "terrible").
- **Arousal (y-axis):** The physiological energy or activation level.
    - *Range:* -1.0 (Torpor, Depression, Calm) to $+1.0$ (Panic, Rage, Excitement).
    - *Extraction Cue:* Punctuation intensity (!), speech velocity (words per minute), and specific lexicon (e.g., "racing," "tired," "explosive").
- **Connection (z-axis):** The degree of relational alignment with self or others.
    - *Range:* -1.0 (Shame, Isolation, Contempt) to $+1.0$ (Belonging, Empathy, Love).
    - *Extraction Cue:* Pronoun usage ("we" vs. "them"), relational prepositions ("with" vs. "at"), and specific emotional contexts defined by Brown (e.g., vulnerability is a gateway to connection).

2.2 The 13 Categories of Emotion: Detection Logic

The Listener must use the following categorical definitions to guide its classification logic. The nuances described here are critical for the Prompt Engineering strategy in the Semantic Processing layer.[1]

2.2.1 Places We Go When Things Are Uncertain or Too Much

- **Emotions:** Stress, Overwhelm, Anxiety, Worry, Avoidance, Excitement, Dread, Fear, Vulnerability.
- **Detection Logic:** These states are characterized primarily by high Arousal ($y > 0.5$).
    - **Overwhelm vs. Stress:** The Listener must distinguish these. "Stress" is being "in the weeds" (high demand, high energy). "Overwhelm" is being "blown" (system failure, inability to process). If the user indicates they cannot function, map to Overwhelm.
    - **Vulnerability:** This is the most critical state for the system. It is defined as uncertainty, risk, and emotional exposure. It is the only state in this cluster that often carries a positive Connection potential ($z > 0$), as it is the prerequisite for intimacy.

2.2.2 Places We Go When We Compare

- **Emotions:** Comparison, Admiration, Reverence, Envy, Jealousy, Resentment, Schadenfreude, Freudenfreude.
- **Detection Logic:** These are relational states defined by a "self-other" differential.
    - **Resentment:** Often misidentified as Anger. The Listener must look for hidden Envy or unmet needs. It is part of the "envy family," not the "anger family."
    - **Schadenfreude:** Pleasure derived from another's misfortune. High Valence ($x > 0$), but deeply negative Connection ($z < -0.8$).

2.2.3 Places We Go When Things Don't Go As Planned

- **Emotions:** Boredom, Disappointment, Expectations, Regret, Discouragement, Resignation, Frustration.
- **Detection Logic:** These represent a "delta" or error function between expectation and reality.
    - **Resignation:** A cessation of effort. Arousal drops to near zero or negative.
    - **Frustration:** High energy ($y > 0.6$) with blocked movement. The user feels "stuck" but energetic.

2.2.4 Places We Go When It's Beyond Us

- **Emotions:** Awe, Wonder, Confusion, Curiosity, Interest, Surprise.
- **Detection Logic:** These are "epistemic emotions."
    - **Awe/Wonder:** Characterized by "self-diminishment" in a positive way. The "I" becomes small, the universe becomes large. High Connection ($z > 0.8$).
    - **Confusion:** The Listener handles this uniquely. Confusion signals high entropy. If detected, the system may prompt the user for clarification rather than forcing a vector.

2.2.5 Places We Go When Things Aren't What They Seem

- **Emotions:** Amusement, Bittersweetness, Nostalgia, Cognitive Dissonance, Paradox, Irony, Sarcasm.
- **Detection Logic:** These are complex, mixed-valence states.
    - **Bittersweetness:** A simultaneous activation of positive and negative Valence. The Listener must report the *resultant* vector (often close to $x=0$) but tag the state metadata as "Mixed."
    - **Nostalgia:** A yearning for the past. Often high Connection to a memory, but disconnection from the present.

2.2.6 Places We Go When We're Hurting

- **Emotions:** Anguish, Hopelessness, Despair, Sadness, Grief.
- **Detection Logic:** Generally low Arousal, low Valence.
    - **Anguish:** Distinct from sadness by its physicality. It is an "excruciating" state. The Listener maps this to the extreme negative bounds of Valence ($x = -1.0$).
    - **Grief:** While painful, Grief is often a function of Love. Therefore, unlike Despair, Grief may maintain a high Connection score ($z > 0.5$) to the lost object.

2.2.7 Places We Go With Others

- **Emotions:** Compassion, Pity, Empathy, Sympathy, Boundaries, Comparative Suffering.
- **Detection Logic:** This cluster defines the z-axis.
    - **Pity vs. Compassion:** As noted, Pity is "near enemy" of Compassion. Pity creates distance ($z < 0$); Compassion bridges it ($z > 0$).
    - **Boundaries:** The Listener must recognize "saying no" as a positive Connection act ("choosing discomfort over resentment").

2.2.8 Places We Go When We Fall Short

- **Emotions:** Shame, Self-Compassion, Perfectionism, Guilt, Humiliation, Embarrassment.
- **Detection Logic:**
    - **Shame:** The master emotion of disconnection. "I am bad." Map to $z = -1.0$.
    - **Guilt:** "I did something bad." Focus is on behavior, not self. Connection remains intact or repairable ($z > -0.2$).

2.2.9 Places We Go When We Search for Connection

- **Emotions:** Belonging, Fitting In, Connection, Disconnection, Insecurity, Invisibility, Loneliness.
- **Detection Logic:**
    - **Belonging vs. Fitting In:** "Fitting In" is assessing the situation and becoming who you need to be to be accepted (External focus, low authenticity). "Belonging" is being who you are (Internal focus, high authenticity). The Listener must parse this distinction carefully.

2.2.10 Places We Go When the Heart Is Open

- **Emotions:** Love, Lovelessness, Heartbreak, Trust, Self-Trust, Betrayal, Defensiveness, Flooding, Hurt.
- **Detection Logic:**
    - **Flooding:** A physiological hijack. The Listener detects this via rapid, incoherent speech or explicit statements of "I can't think." This triggers a system "pause" state.

2.2.11 Places We Go When Life Is Good

- **Emotions:** Joy, Happiness, Calm, Contentment, Gratitude, Foreboding Joy, Relief, Tranquility.
- **Detection Logic:**
    - **Foreboding Joy:** The rapid shift from Joy to Fear ("It's too good to be true"). High Valence tainted by High Arousal/Anxiety.
    - **Gratitude:** The antidote to Foreboding Joy.

2.2.12 Places We Go When We Feel Wronged

- **Emotions:** Anger, Contempt, Disgust, Dehumanization, Hate, Self-Righteousness.
- **Detection Logic:** The "Action" emotions.
    - **Anger:** A catalyst. High Arousal. Can be constructive ($z > -0.2$) or destructive ($z < -0.8$).
    - **Dehumanization:** The absolute zero of Connection ($z = -1.0$).

2.2.13 Places We Go to Self-Assess

- **Emotions:** Pride, Hubris, Humility.
- **Detection Logic:**
    - **Hubris:** Inflated pride that disconnects ($z < 0$).
    - **Humility:** Grounded confidence ($z > 0$).

---

3. System Architecture: The Listener Module

The Listener is designed as a microservice within the broader L.O.V.E. stack. It interfaces with the Client (Experience), the Database (Observer), and the Math Engine (Versor).

3.1 High-Level Topology

The architecture must solve the tension between latency (immediate feedback) and depth (accurate psychological mapping). We employ a "Hybrid Edge-Cloud" topology.

Component	Location	Technology	Responsibility
Listener Edge	Mobile Device	React Native, whisper.rn	Real-time audio capture, local transcription,

			immediate UI feedback.
Listener Gateway	Cloud (API)	FastAPI, Nginx	Authentication, Rate Limiting, Request Orchestration.
Listener Queue	Cloud (Memory)	Redis, Arq	Asynchronous task management, buffering.
Listener Brain	Cloud (Compute)	faster-whisper, LangChain	High-fidelity transcription, Semantic Vectorization, PII Sanitization.

3.2 Tier 1: The Edge (Client-Side Ingestion)

The "Experience" application runs on React Native (targeting iOS and Android). The Listener's edge component is a background service within this app.

3.2.1 Audio Capture & Local Transcription

- **Library:** The client shall utilize whisper.rn.[1] This is a React Native binding for whisper.cpp, allowing OpenAI's Whisper model to run directly on the device CPU/Neural Engine.
- **Model Selection:** The system must download the quantized ggml-tiny.en.bin model (approx 75MB) upon first launch. This model offers the optimal balance of speed vs. accuracy for real-time feedback.
- **CoreML/NNAPI:** The implementation must enable enableCoreML: true (iOS) and enableNNAPI: true (Android) in the initialization config to offload inference to the hardware accelerators (Neural Engine / DSP). This is critical to prevent the main JS thread from freezing during transcription.[7]
- **New Architecture (Fabric) Constraint:** As of 2025, React Native is transitioning to the "New Architecture" (Fabric). While Expo SDK 52 supports this, specific native modules like whisper.rn may exhibit instability.
    - *Mitigation:* The development plan must verify whisper.rn compatibility. If Fabric issues arise (e.g., synchronous bridge failures), the specific transcription component should be isolated, or the app configuration in gradle.properties must set newArchEnabled=false temporarily until the library is patched.[8]

3.2.2 The Optimistic UI Pattern

User trust is established through responsiveness. The Edge Listener must implement an "Optimistic Update" pattern:

1. User speaks -> whisper.rn generates partial text stream.
2. Client immediately renders this text to the screen.

3. Client performs a crude, local sentiment analysis (using a lightweight library like vader-sentiment-js or a simple lexicon lookup) to tint the "Soul Sphere" visualization immediately (e.g., turn red if "angry" words are detected).
4. This provides immediate feedback (< 200ms) while the high-fidelity cloud process runs in the background.

3.3 Tier 2: The Cloud (Server-Side Processing)

The core intelligence resides in the cloud. This layer is responsible for the "Canonical Truth"—the data that gets stored in the Observer.

3.3.1 The Ingestion API (FastAPI)

The backend is built on **FastAPI** due to its native asynchronous support, which is essential for handling long-lived audio processing requests without blocking.[11]

- **Endpoint:** POST /listener/ingest
- **Payload:** multipart/form-data containing the audio file (or text) and metadata.
- **Audio Normalization:** Mobile audio comes in various formats (.m4a, .aac, .opus). The API must use ffmpeg-python to normalize all incoming audio to **16kHz, Mono, WAV format** before passing it to the inference engine. This standardizes the input for Whisper.[12]

3.3.2 The Asynchronous Queue (Arq + Redis)

Processing audio is CPU/GPU intensive. We cannot process this within the HTTP request/response cycle.

- **Architecture:** We utilize **Arq** (Async Redis Queue) over Celery.
- **Reasoning:** Celery is heavyweight and historically synchronous-first. Arq is built on asyncio and redis, making it a native fit for FastAPI. It handles job deferral, retries, and result persistence with significantly less overhead.[13]
- **Flow:**
    1. API receives audio -> Pushes job to Redis (job_id).
    2. API returns 202 Accepted with task_id.
    3. Worker node picks up job -> Runs faster-whisper.
    4. Client polls or receives WebSocket push upon completion.

3.3.3 High-Fidelity Transcription (Faster-Whisper)

- **Engine:** The worker nodes shall run faster-whisper (based on CTranslate2). This is up to 4x faster than the standard OpenAI Whisper implementation while using less VRAM.[16]
- **Model:** large-v3. This model is required to capture the nuance of emotional speech, including pauses, dysfluencies ("um...", "uh..."), and intonation markers that might imply hesitation (uncertainty) or rapid-fire delivery (anxiety).
- **VAD (Voice Activity Detection):** The configuration must enable vad_filter=True to strip non-speech segments, preventing the model from hallucinating text during silence.[16]

---

4. Semantic Processing Engine: The "Brain"

Once text is transcribed, it is merely data. The Listener must transmute it into *meaning*. This is the domain of the Semantic Processing Engine.

4.1 Orchestration: LangChain

We utilize **LangChain** to orchestrate the interaction with the Large Language Model (LLM). LangChain provides the necessary abstractions for prompt management, output parsing, and error handling.[1]

- **Model Provider:** The system should be agnostic but optimized for **Llama 3 (70B)** via Groq (for speed) or **GPT-4o** (for reasoning depth). The requirement for "detailed software requirements" suggests we need the reasoning capabilities of a larger model to parse complex emotional states like "Schadenfreude" or "Foreboding Joy."

4.2 The Psychometric Prompt Strategy

The quality of the VAC vector depends entirely on the prompt. We cannot simply ask "What emotion is this?" We must guide the LLM through a psychometric evaluation.

System Prompt Design:

"You are the Listener, an expert psychometrician trained in the work of Dr. Brené Brown's *Atlas of the Heart*. Your task is to analyze the user's input and map it to the 3-dimensional VAC Model (Valence, Arousal, Connection) and identifying the specific emotional state from the 87 defined categories."

Chain-of-Thought (CoT) Logic:

To ensure accuracy, the prompt must enforce a "step-by-step" reasoning process before generating the final JSON:

1. **Analyze Valence:** Look for hedonic keywords. Is the user suffering or thriving?
2. **Analyze Arousal:** Look for energy markers. Is the user manic, calm, or depressive?
3. **Analyze Connection (The Critical Step):** Look for relational vectors. Is the user moving *toward*, *away*, or *against* others? (e.g., "I want to be alone" = Solitude (+Connection to self) OR Isolation (-Connection to others). Context determines the vector).
4. **Select Category:** Which of the 13 *Atlas* categories does this fit?
5. **Select Emotion:** Pick the precise label from the 87.

4.3 Structured Output Enforcement (Pydantic)

The Versor (math engine) requires strict floating-point numbers. It cannot handle "I think it's mostly happy." We must enforce a strict schema.

- **Tool:** LangChain's PydanticOutputParser or the model's native "Structured Output" mode (e.g., OpenAI JSON mode).[17]
- Schema Definition:
  The Listener must implement the following Pydantic models to validate LLM output:

Python

```
from pydantic import BaseModel, Field, validator
from typing import Literal

class VACVector(BaseModel):
    valence: float = Field(..., ge=-1.0, le=1.0, description="Pleasure vs Displeasure")
    arousal: float = Field(..., ge=-1.0, le=1.0, description="Energy vs Lethargy")
    connection: float = Field(..., ge=-1.0, le=1.0, description="Relational Alignment")

class EmotionalClassification(BaseModel):
    primary_emotion: str = Field(..., description="One of the 87 Atlas emotions")
    category: str = Field(..., description="One of the 13 Atlas categories")
    vac: VACVector
    confidence: float = Field(..., ge=0.0, le=1.0)
    reasoning: str = Field(..., description="Brief psychometric analysis")

    @validator('primary_emotion')
    def validate_emotion(cls, v):
        # Check against the static list of 87 emotions
        if v not in ATLAS_EMOTIONS_LIST:
            raise ValueError(f"{v} is not a valid Atlas emotion")
        return v
```

4.4 The "Connection" Axis Disambiguation

This is the most significant technical challenge. Standard sentiment models blend Valence and Connection.

- *Scenario:* "I feel so jealous of her success."
- *Standard Model:* Negative Valence. High Arousal.
- *L.O.V.E. Model:* Negative Valence. High Arousal. **Negative Connection** (Comparison/Envy).
- *Scenario:* "I am so proud of her success."
- *Standard Model:* Positive Valence. High Arousal.
- *L.O.V.E. Model:* Positive Valence. High Arousal. **Positive Connection** (Freudenfreude).

The Listener must be explicitly trained (via few-shot prompting) to distinguish these. The prompt must include examples like:

- *Input:* "I feel sorry for him." -> *Label:* Pity (Connection: -0.5).
- *Input:* "I feel his pain." -> *Label:* Compassion (Connection: +0.8).

---

5. Privacy, Security, and Data Sanitization

Given the deeply personal nature of the data, the Listener must implement a "Privacy by Design" philosophy.

5.1 PII Sanitization (The Scrubbing Layer)

Before any text is stored in the user_trajectory table (Observer) or used for long-term vector embeddings, it must be sanitized.

- **Mechanism:** The Listener pipeline includes a Named Entity Recognition (NER) step using **Spacy** (en_core_web_sm) or a specialized PII-stripping LLM pass.
- **Entities to Scrub:** PERSON, ORG (Organization), GPE (Location), DATE.
- **Replacement Strategy:** Replace with tokens: [NAME], ``, [LOC].
- **Order of Operations:**
    1. Transcription (Raw).
    2. Semantic Analysis (Raw - context needed for emotion).
    3. **Sanitization** (The Scrub).
    4. Storage (Sanitized).
    - *Note:* The raw audio and raw transcript are ephemeral. They exist only in the Redis queue and worker memory during processing. Once the vector is extracted and the sanitized log created, the raw data is effectively destroyed (unless the user explicitly opts into "Journal Mode" with enhanced encryption).

5.2 HIPAA/GDPR Considerations

- **Encryption at Rest:** The PostgreSQL database (Observer) must utilize Transparent Data Encryption (TDE).
- **Encryption in Transit:** All API communication uses TLS 1.3.
- **Data Minimization:** The Listener does not retain audio files. Audio is processed and discarded immediately.

---

6. Detailed Interface Specifications

The following section outlines the specific API contracts required for the Listener.

6.1 API Endpoint: POST /ingest

Description: The primary entry point for audio/text ingestion.
Request:

- **Headers:** Authorization: Bearer <JWT>, Content-Type: multipart/form-data
- **Body:**
    - audio: (Binary, Optional) The audio file.
    - text: (String, Optional) Direct text input.
    - timestamp: (ISO8601) Client time.

- client_id: (UUID) Anonymized device ID.

Response (202 Accepted):

JSON

```
{
  "status": "queued",
  "job_id": "job_8f7d6a5e",
  "queue_position": 1,
  "estimated_wait": 1.2
}
```

6.2 WebSocket Stream: /ws/listener/{client_id}

Description: A persistent connection for pushing real-time updates to the Experience module.

Protocol:

1. **Client Connects.**
2. **Server Sends JOB_STARTED:** "Processing audio..."
3. **Server Sends INTERIM_TRANSCRIPT:** (Optional, if streaming backend enabled).
4. **Server Sends ANALYSIS_COMPLETE:**

   JSON

   ```
   {
     "type": "ANALYSIS_COMPLETE",
     "payload": {
       "transcript": "I felt really overwhelmed at the meeting...",
       "sanitized_transcript": "I felt really overwhelmed at the...",
       "emotion": {
         "label": "Overwhelm",
         "category": "Places We Go When Things Are Uncertain or Too Much",
         "vac": { "x": -0.6, "y": 0.8, "z": -0.4 }
       },
       "insight": "High arousal detected. System suggests grounding exercise."
     }
   }
   ```

6.3 Integration with Observer (Database)

The Listener does not write directly to the database. It calls an internal service method provided by the Observer module.

- **Observer Interface:** Observer.log_state(user_id, vac_vector, quaternion, metadata)
- **Data Handoff:**

    - The Listener computes the VAC.
    - The Listener *invokes* the Versor to get the Quaternion (q).
    - The Listener bundles (VAC, q, Sanitized Text, Emotion Label) and sends to Observer.

---

7. Implementation Roadmap

Phase 1: The Core Ingestion (Weeks 1-2)

- **Objective:** Establish the FastAPI shell and the Edge transcription.
- **Tasks:**
    1. Setup FastAPI project with Poetry.
    2. Implement POST /ingest dummy endpoint.
    3. Create React Native POC with whisper.rn to verify Fabric compatibility.
    4. Implement basic Audio -> text pipeline using faster-whisper.

Phase 2: The Semantic Brain (Weeks 3-4)

- **Objective:** Integrate LLM and Atlas Ontology.
- **Tasks:**
    1. Implement LangChain structure.
    2. Create the EmotionalClassification Pydantic model.
    3. Develop the "Psychometric System Prompt" with few-shot examples.
    4. Run a test battery of 100 text samples to calibrate VAC output against manual tagging.

Phase 3: The Async Queue & Optimization (Weeks 5-6)

- **Objective:** Scalability and Latency.
- **Tasks:**
    1. Deploy Redis and Arq workers.
    2. Implement VAD filtering and Audio Normalization (ffmpeg).
    3. Optimize the Edge-to-Cloud handoff (upload chunking).
    4. Implement WebSocket push notifications.

---

8. Conclusion

The Listener is the foundation of Project L.O.V.E. It is tasked with the immensely difficult job of translating the messy, analog reality of human speech into the precise, digital language of quaternions. By combining edge-based responsiveness with cloud-based semantic depth, and by adhering to the rigorous ontology of *Atlas of the Heart*, the Listener ensures that every rotation of the "Soul Sphere" is grounded in genuine emotional truth.

This specification provides the exhaustive detail required to build this module. It addresses the theoretical nuance of the "Connection" axis, the architectural constraints of mobile development, and the security imperatives of affective data. The system is now ready for implementation.

9. Appendix: Data Tables

Table 1: Representative VAC Mappings for Testing

Category	Emotion	Valence (x)	Arousal (y)	Connection (z)	Reason
Uncertainty	**Overwhelm**	-0.6	+0.9	-0.3	High energy saturation, frayed connection.
Comparison	**Resentment**	-0.5	+0.6	-0.5	Anger mixed with Envy (disconnect).
Hurting	**Grief**	-0.9	-0.4	+0.5	Deep pain, but connection to the lost remains.
Open Heart	**Flooding**	-0.4	+1.0	-0.8	Physiological hijack, total disconnect.
Connection	**Belonging**	+0.8	+0.4	+1.0	The definition of max positive connection.
Self-Assess	**Hubris**	+0.7	+0.6	-0.8	Pride that elevates self above others.

Table 2: Tech Stack Summary

Component	Choice	Justification
Mobile API	React Native (Expo)	Cross-platform, rich ecosystem.
Edge ASR	whisper.rn (whisper.cpp)	Offline capability, zero latency UI.
Backend API	FastAPI (Python)	Async native, high performance.
Task Queue	Arq + Redis	Lightweight async job management.

Cloud ASR	faster-whisper	4x faster than standard Whisper, low VRAM.
LLM Orchestrator	LangChain	Robust prompt management & output parsing.
Validation	Pydantic	Strict schema enforcement for Math engine.

(End of Requirements Document)

Works cited

1. L.O.V.E. Project Software Requirements.pdf
2. Brene Brown Emotions, accessed December 2, 2025, https://www.depts.ttu.edu/rise/Posters/BreneBrownEmotions.pdf
3. 87 Human Emotions and Experiences - 1page | PDF - Scribd, accessed December 2, 2025, https://www.scribd.com/document/656659375/87-Human-Emotions-and-Experiences-1Page
4. 87 Human Emotions & Experiences - Brene Brown, accessed December 2, 2025, https://brenebrown.com/wp-content/uploads/2022/03/AtlasoftheHeartListofEmotions.pdf
5. Book Summary – Atlas of the Heart: Mapping Meaningful Connection and the Language of Human Experienc - Readingraphics, accessed December 2, 2025, https://readingraphics.com/book-summary-atlas-of-the-heart/
6. Atlas of the heart, accessed December 2, 2025, https://cdn.prod.website-files.com/675455180fd2f454e6cbf05c/681eb6f2b5c64d8e149f2ccb_66160125184.pdf
7. mybigday/whisper.rn: React Native binding of whisper.cpp. - GitHub, accessed December 2, 2025, https://github.com/mybigday/whisper.rn
8. React Native's New Architecture - Expo Documentation, accessed December 2, 2025, https://docs.expo.dev/guides/new-architecture/
9. Upgraded to Expo SDK 52 — Will the New Architecture Help with Heavy JS Loops & Native Module Tasks? : r/reactnative - Reddit, accessed December 2, 2025, https://www.reddit.com/r/reactnative/comments/1m4j5er/upgraded_to_expo_sdk_52_will_the_new_architecture/
10. How is the new Fabric architecture changing React Native performance compared to native apps? : r/reactnative - Reddit, accessed December 2, 2025, https://www.reddit.com/r/reactnative/comments/1ou2pbo/how_is_the_new_fabric_architecture_changing_react/
11. Concurrency and async / await - FastAPI, accessed December 2, 2025, https://fastapi.tiangolo.com/async/
12. Real-Time Audio Processing with FastAPI & Whisper: Complete Guide 2024, accessed December 2, 2025, https://trinesis.com/blog/articles-1/real-time-audio-processing-with-fastapi-whisper-complete-guide-2024-70

13. Truly Asynchronous Task Queue in FastAPI Using Arq - YouTube, accessed December 2, 2025, https://www.youtube.com/watch?v=4Xj_6N708Nc
14. What's the difference between FastAPI background tasks and Celery tasks? - Stack Overflow, accessed December 2, 2025, https://stackoverflow.com/questions/74508774/whats-the-difference-between-fastapi-background-tasks-and-celery-tasks
15. Background tasks in Python : ARQ VS Celery - YouTube, accessed December 2, 2025, https://www.youtube.com/watch?v=OY1Yi1cE4sg
16. Faster Whisper transcription with CTranslate2 - GitHub, accessed December 2, 2025, https://github.com/SYSTRAN/faster-whisper
17. Clarification on how Pydantic schema descriptions are used in with_structured_output, accessed December 2, 2025, https://forum.langchain.com/t/clarification-on-how-pydantic-schema-descriptions-are-used-in-with-structured-output/1612
18. LangChain Structured Outputs: A Guide to Tools and Methods - Mirascope, accessed December 2, 2025, https://mirascope.com/blog/langchain-structured-output